

# Rate Limiting & Throttling

Phase 2 · Intermediate · Estimated time: 1.5 hours

---

## 1. Concept From First Principles

Chapter 8's message queues handle bursts by letting work queue up and drain gradually. But some resources — a login endpoint, a third-party API you're paying per-call for, a database under heavy write load — can't simply be allowed to queue indefinitely; they need a hard cap on how much traffic gets through in the first place. That's **rate limiting**: capping the number of requests a client (a user, an API key, an IP address) can make within a given time window, and rejecting or delaying requests beyond that cap.

Rate limiting serves several distinct purposes worth naming separately, because the right implementation can depend on which one you're actually solving for. It protects backend resources from being overwhelmed by legitimate-but-excessive traffic (a buggy client polling too aggressively). It defends against abuse and denial-of-service attempts. It enforces business rules (a free-tier API plan capped at 1,000 requests/day). And it protects downstream dependencies with their own hard limits — if a third-party SMS or payment provider caps you at 100 calls/second, your own system needs to self-limit to that rate before it ever gets there, or every call beyond the limit simply fails anyway, just wastefully.

### Backend engineer analogy

This is the server-side mirror of Laravel's built-in **throttle** middleware (`Route::middleware('throttle:60,1')`) or Express's **express-rate-limit** package — if you've ever capped an API route to 60 requests per minute per user, you've already implemented a rate limiter. This chapter goes into exactly which algorithm those middleware packages use under the hood, and why the choice matters at scale.

### Token bucket

The **token bucket** algorithm maintains a bucket with a fixed capacity of tokens. Tokens are added to the bucket at a steady, configured rate (say, 10 tokens/second) up to the bucket's maximum capacity. Every incoming request must consume one token to proceed; if the bucket is empty, the request is rejected (or queued, depending on design). The key property: because tokens accumulate during idle periods up to the bucket's capacity, a client that hasn't made requests in a while can burst up to that capacity all at once, then falls back to the steady replenishment rate. This makes token bucket the standard choice for consumer-facing APIs, since it tolerates natural, bursty usage patterns (a user rapid-firing a few requests, then going idle) without feeling punitive.

### Leaky bucket

The **leaky bucket** algorithm flips the framing: incoming requests fill a bucket (queue), and the bucket "leaks" (processes requests) at a constant, fixed rate, regardless of how bursty the input was. If the bucket overflows (too many requests arrive before it can drain), excess requests are dropped. This

produces smooth, constant-rate output no matter how spiky the input was, which is exactly what you want when protecting a downstream system that genuinely cannot handle bursts, even short ones — video transcoding pipelines and payment processors are commonly cited examples where a strictly even output rate matters more than accommodating client burstiness.

## Sliding window and fixed window counters

A simpler alternative is the **fixed window counter**: count requests in discrete time buckets (e.g., "this calendar minute"), reset the counter to zero at each window boundary, and reject once the count hits the limit. It's cheap to implement but has a known edge-case flaw: a client can send the maximum allowed requests in the last moment of one window and immediately the maximum again in the first moment of the next window, effectively doubling the intended rate in a short burst spanning the boundary. The **sliding window** approach fixes this by counting requests within a continuously moving time window (the last 60 seconds from right now, not "this calendar minute"), giving a smoother, more accurate rate limit at the cost of slightly more bookkeeping.

At scale, rate limiting itself has to be a distributed system's problem, not a single-server one. If your API runs behind a load balancer across 20 app-server instances, each server independently tracking "has this user hit 60 requests this minute" in its own local memory would let a user get 60 requests through *each* server, effectively multiplying their real limit by 20. The standard fix is a shared, centralized counter store — typically Redis, using atomic increment operations (INCR with an expiry) — so every app server checks and updates the same, single source of truth for each client's current usage.

## 2. Real-World Example / Case Study

Razorpay's and PhonePe's public payment APIs enforce per-merchant rate limits (often documented explicitly as "N requests per second") using token-bucket-style limiting, allowing merchants brief legitimate bursts (a flash sale causing many near-simultaneous checkout calls) while still protecting the platform's core infrastructure from any single merchant's traffic pattern overwhelming shared capacity.

Swiggy's and Zomato's OTP/login endpoints are classic candidates for strict, low-limit rate limiting — capping OTP requests per phone number to something like 3–5 per hour specifically defends against SMS-bombing abuse and cost (each SMS costs money to send) rather than protecting raw server capacity, illustrating that rate limiting's purpose isn't always about backend load.

Careem and Noon, integrating with third-party services like SMS gateways, mapping APIs, or payment processors that themselves enforce hard rate limits, need their own internal rate limiting to stay under those external caps — self-throttling proactively rather than discovering the limit reactively through a wave of failed external API calls during a traffic spike, which would be a far worse failure mode.

## 3. Diagram

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Token bucket (allows bursts):</p> <p>Capacity: 10 tokens<br/>Refill: 2 tokens/sec</p> <p>[***. . . .] &lt;- 3 tokens used,<br/>refills over time</p> <p>Idle client -&gt; bucket fills to 10<br/>-&gt; next burst of up to 10 allowed<br/>instantly, then throttled</p> <p>Distributed rate limiting (shared state):</p> <p>[App-1] [App-2] [App-3] --all check/update--&gt; [ Redis: INCR user:42 ]<br/>Without shared state: each server allows its OWN limit -&gt; N x over-limit.</p> | <p>Leaky bucket (smooths output):</p> <p>Requests arrive (bursty):<br/>R R R R R R</p> <p> </p> <p>[ Queue/Bucket ]</p> <p> </p> <p>leaks at fixed rate: 1/sec<br/>R -&gt; R -&gt; R -&gt; R -&gt; R -&gt; R<br/>(excess overflow = dropped)</p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 4. Trade-offs Table

| Algorithm            | Handles bursts                     | Output smoothness         | Typical use                             |
|----------------------|------------------------------------|---------------------------|-----------------------------------------|
| Token bucket         | Yes, up to bucket capacity         | Variable (bursty allowed) | Consumer APIs, general-purpose limiting |
| Leaky bucket         | No — smooths everything out        | Constant, fixed rate      | Protecting fragile downstream systems   |
| Fixed window counter | Edge-case double-burst at boundary | Simple but imprecise      | Quick, low-effort implementations       |
| Sliding window       | Accurate, no boundary exploit      | Smooth and precise        | Production APIs needing strict accuracy |

## 5. Common Interview Questions

### Q1: "Design a rate limiter for a public API. Walk through your algorithm choice."

Model approach: Recommend token bucket as the default for consumer-facing APIs (tolerates natural burst patterns), explain the refill-rate and capacity parameters, and note that a sliding window would be the choice instead if strict, exploit-free accuracy matters more than burst tolerance.

### Q2: "Your rate limiter works fine on one server but fails once you scale to multiple servers. Why, and how do you fix it?"

Model approach: Diagnose the problem as each server tracking its own local counter, effectively multiplying the real limit by the number of servers, and fix it with a centralized, shared counter store (Redis, using atomic INCR + expiry) that every server checks against.

### Q3: "What's the difference between token bucket and leaky bucket, concretely?"

Model approach: Token bucket allows accumulated capacity to be spent in a burst; leaky bucket forces a constant output rate regardless of input burstiness. Give a concrete example of each: token bucket for a user-facing API, leaky bucket for a video-processing pipeline that can't handle uneven load.

### Q4: "A client is rate-limited. What should the API return, and what should the client do?"

Model approach: Return HTTP 429 Too Many Requests, along with headers indicating the limit, remaining quota, and when it resets (commonly Retry-After or X-RateLimit-\* headers), so well-behaved clients can back off and retry appropriately instead of hammering the endpoint further.

## 6. Practice Exercises

- 1 Design a rate limiter for an OTP-send endpoint that must allow at most 3 requests per phone number per hour. Choose an algorithm and justify it.
- 2 Explain, with a concrete number example, why a fixed window counter can let a client send up to 2x its intended limit across a window boundary.
- 3 Your app calls a third-party SMS API capped at 50 requests/second. Design an internal rate limiter to stay safely under that cap during a traffic spike.

## 7. Curated YouTube Videos

| Language | Video & Channel                                                        | Why it helps                                                           |
|----------|------------------------------------------------------------------------|------------------------------------------------------------------------|
| English  | "System Design Interview: Rate Limiting Algorithms That Get You Hired" | Covers token bucket, leaky bucket, fixed and sliding window with code. |
| English  | "Master Rate Limiting - System Design"                                 | Broad walkthrough of rate limiting strategies and where each fits.     |
| Hindi    | "System Design Complete Course 2026" (Hindi + English, 10+ hrs)        | Dedicated module on rate limiting as part of a full bilingual course.  |
| Hindi    | "Complete System Design Roadmap 2025   HLD & LLD" — CodeHelp           | Places rate limiting in context of the broader Hindi HLD roadmap.      |

## 8. Key Takeaways

- Rate limiting caps how much traffic a client can send in a given window, protecting resources and enforcing business rules.
- Token bucket allows bursts up to a capacity, then throttles to a steady refill rate — good for consumer APIs.
- Leaky bucket forces a constant output rate regardless of input burstiness — good for fragile downstream systems.
- Fixed window counters are simple but have a known boundary-exploit flaw; sliding windows fix this at some extra cost.
- Rate limiting must use shared, centralized state (e.g. Redis) once you run more than one app server.
- Rate-limited responses should return 429 with clear retry-after information for well-behaved clients.
- Self-imposed internal rate limits protect you from exceeding third-party APIs' own hard limits.